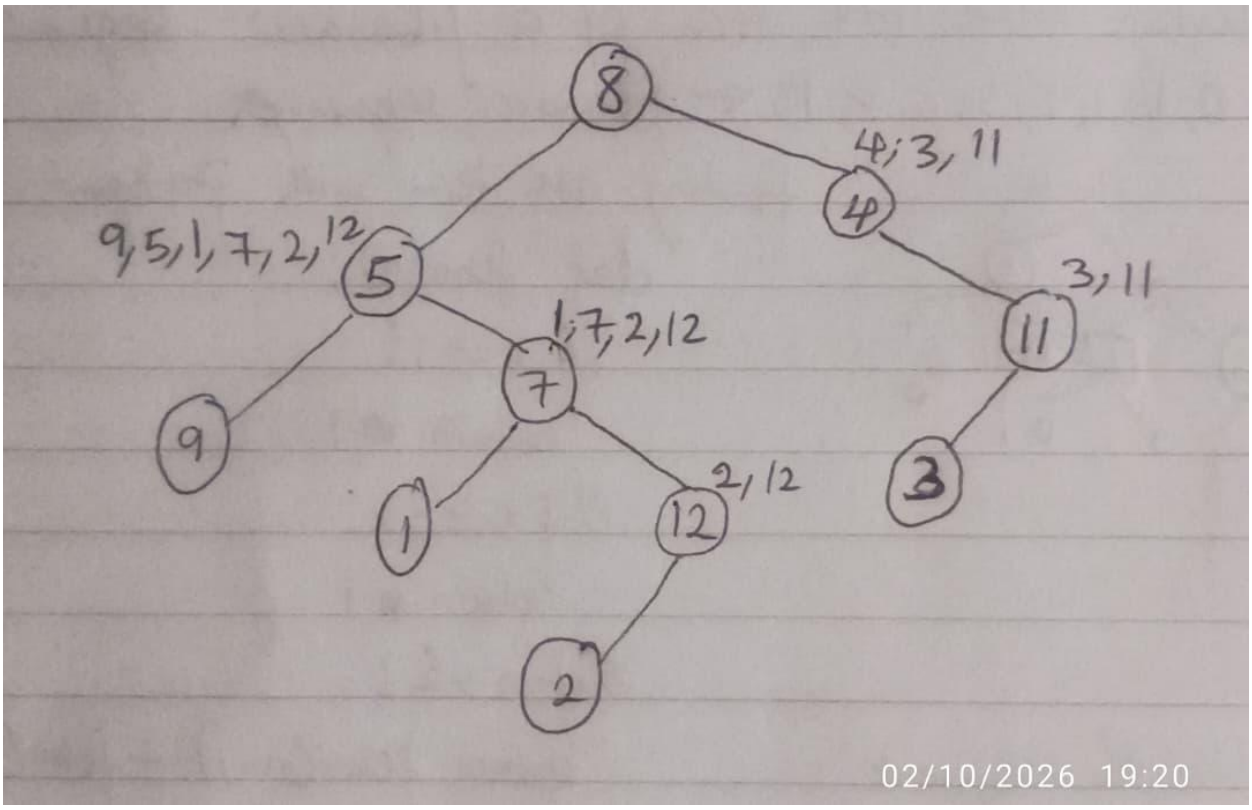
Reconstruct a Tree with is In-order and Post-order Traversal Results

1. Question (1) to solve (20%)
   Given the sequence of in-order traversal for a general binary tree {9, 5, 1, 7, 2, 12, 8, 4, 3, 11}, and given the sequence of post-order traversal for the same tree { 9, 1, 2, 12, 7, 5, 3, 11, 4, 8}, please explain how to reconstruct the binary tree and draw its graphical representation in your answer. Please include step by step explanations to guarantee partial credits. Note that this tree is a general binary tree, but not a BST.
   Hint: thinking about the definition of in-order and post-order traversal. For in-order traversal, after its left subtree is visited, then the root is visited. This is true for any subtrees of any node. For post-order traversal, the root is always outputted at last, after its left subtree and right subtree are both visited. This is true for any subtrees of any node in the tree.

Solution



post-order traversal: { 9, 1, 2, 12, 7, 5, 3, 11, 4, 8}

- Left subtree
- Right subtree
- Root

in-order traversal: {9, 5, 1, 7, 2, 12, 8, 4, 3, 11}

- Left subtree
- Root
- Right subtree

Steps:

1. The post-order traversal visits nodes in the order of: [Left subtree, Right subtree, Root]. With this we can determine our Root, that is the last element in the array, which is 8.
2. Then we need to determine what is the left and right subtree of our [8], we need to locate the 8 in the in-order array then check what are the elements at its left and its right.

   [9, 5, 1, 7, 2, 12]8 [4, 3, 11]

3. To determine the node at the left hand side of 8: we will scan the post order from right to left, the element we see first that is also present amongst this [9, 5, 1, 7, 2, 12] elements will be the node at 8s left hand side: and that is 5.
   4. Then we need to determine what is the left and right subtree of our [5], we need to locate the 5 in the in-order array then check what are the element at its left and its right.

      [9]5[1, 7, 2, 12]

   5. To determine the node at the left hand side of 5: we will scan the post order from right to left, the element we see first that is also present amongst this [9] elements will be the node at 5s left hand side: and that is 9. And we can safely pick 9 since that is the only element on the left hand side.
   6. To determine the node at the right hand side of 5: we will scan the post order from right to left, the element we see first that is also present amongst this [1, 7, 2, 12] elements will be the node at 5s right hand side: and that is 7.
   7. Then we need to determine what is the left and right subtree of our [7], we need to locate the 7 in the in-order array then check what are the element at its left and its right.

      [1] 7[2, 12]

8. To determine the node at the left hand side of 7: we will scan the post order from right to left, the element we see first that is also present amongst this [1] elements will be the node at 7s left hand side: and that is 1. And we can safely pick 1 since that is the only element at the left hand side.

9. To determine the node at the right hand side of 7: we will scan the post order from right to left, the element we see first that is also present amongst this [2, 12] elements will be the node at 7s right hand side: and that is 12.
10. Then we need to determine what is the left and right subtree of our [12], we need to locate the 12 in the in-order array then check what are the element at its left and its right.

    [2] 12

11. so, 2 will be to the left hand side of our 12.

    Remember we still have array at the right-hand side of 8[4,3,11]

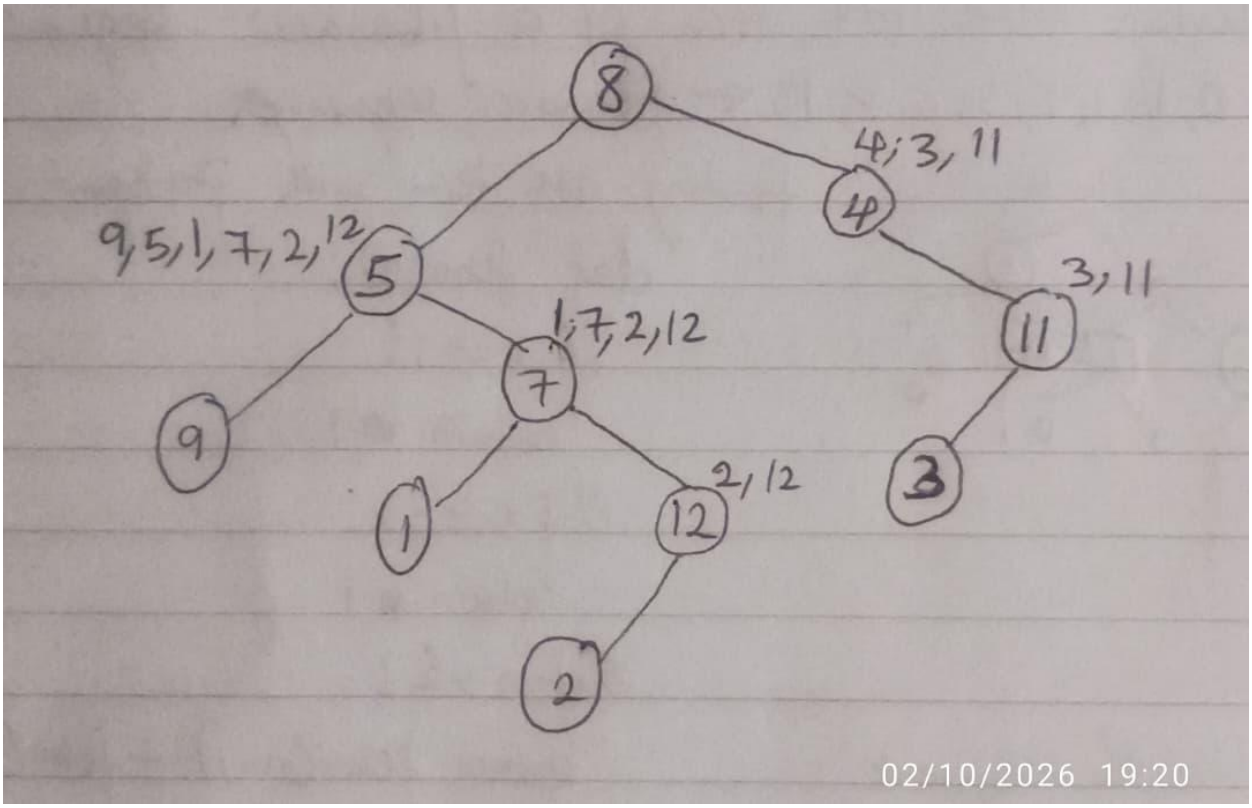
    12. So we need to determine the node at the right hand side of 8: we will scan the post order from right to left, the element we see first that is also present amongst these elements will be the node at 8s right hand side: and that is 4.
    13. Then we need to determine what is the left and right subtree of our [4], we need to locate the 4 in the in-order array then check what are the element at its left and its right.

        4 [3, 11]

    14. Since we only have elements at the right hand side alone, we will only have to develop the right hand side. To determine the node at the right handside of 4: we will scan the post order from right to left, the element we see first that is also present amongst this [3,11] elements will be the node at 4s right hand side: and that is 11.
15. Then we need to determine what is the left and right subtree of our [11], we need to locate the 11 in the in-order array then check what are the element at its left and its right.

    [3] 11

16. Since there is only one value on the left hand side, then we will be developing the left hand side. To determine the node at the left hand side of 11: we will scan the post order from right to left, the element we see first that is also present amongst this [3] elements will be the node at 11s left hand side: and that is 3. And we can safely pick 3 since that is the only element on the left hand side.



Question 2.

The code is present in this same folder as program.cs

```
PS C:\Users\akola\Desktop\Temitope\Adv Algorithm> dotnet run
          BINARY TREE RECONSTRUCTION TEST

 Input In-Order Sequence:   9, 5, 1, 7, 2, 12, 8, 4, 3, 11
 Input Post-Order Sequence: 9, 1, 2, 12, 7, 5, 3, 11, 4, 8

 Reconstructed In-Order Traversal:
 9, 5, 1, 7, 2, 12, 8, 4, 3, 11
 Reconstructed Post-Order Traversal:
 9, 1, 2, 12, 7, 5, 3, 11, 4, 8

 VERIFICATION RESULT:
 Both traversals match the input sequences exactly.
PS C:\Users\akola\Desktop\Temitope\Adv Algorithm>
```